

Politechnika Świętokrzyska w Kielcach Wydział Elektrotechniki Automatyki i Informatyki		
Projekt zespołowy		
Temat: Kamera bezpieczeństwa esp32		
Wykonali: Adrian Rewera Piotr Kozłowski Kamil Raczyński	Grupa: 3ID15A	Data oddania projektu: 25.05.2026

1. Cel projektu

Głównym celem projektu było stworzenie nowoczesnego systemu monitoringu wizyjnego opartego na mikrokontrolerze wbudowanym. Zaprojektowane rozwiązanie różni się od klasycznych, statycznych kamer IP tym, że pozwala na zdalną, fizyczną zmianę obszaru monitorowanego za pomocą serwomechanizmu, zachowując przy tym płynność działania aplikacji webowej. System na bieżąco strumieniuje obraz z kamery bezpośrednio do przeglądarki użytkownika, a także pozwala na natychmiastową reakcję. Za pomocą dedykowanych przycisków w panelu graficznym administrator może płynnie obracać kamerą w lewo i w prawo.

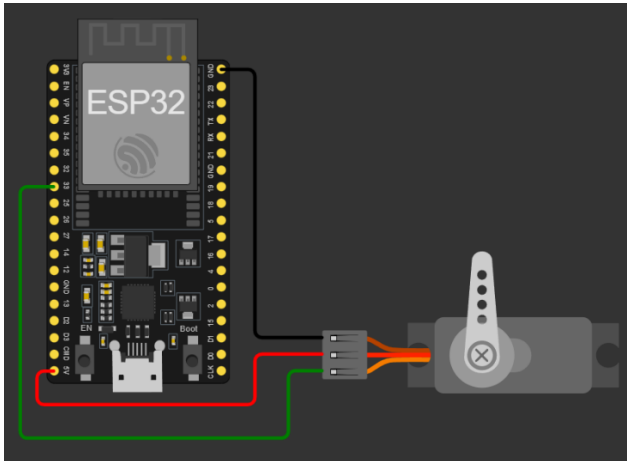
2. Wykorzystane komponenty i technologie

Urządzenie bazuje na łatwo dostępnych elementach elektronicznych, z których najważniejsze to:

- **Płytką główną:** ESP32-WROWER-DEV, odpowiedzialna za łączność Wi-Fi, przetwarzanie logiki oraz hostowanie serwera WWW.
- **Moduł wizyjny:** Kamera OV3660 z matrycą rejestrującą obraz.
- **Element wykonawczy:** Serwomechanizm SG90, umożliwiający fizyczny obrót kamery (zakres obrotu do 180 stopni).

System oprogramowania został napisany w języku C/C++ (zgodnym ze środowiskiem Arduino IDE) i korzysta z dedykowanych bibliotek takich jak WiFi.h, esp_camera.h, esp_http_server.h oraz ESP32Servo.h do zarządzania poszczególnymi blokami aplikacji. Zastosowano również technologię szybkiej konfiguracji ESP BLE Prov.

3. Schemat podłączenia



4. Logowanie, wyświetlanie strony i autoryzacja

Zamiast ogólnodostępnego podglądu, wprowadzono warstwę zabezpieczeń chroniącą przed nieautoryzowanym dostępem. Posiada ona własne trasy HTTP. Funkcja `index_handler` decyduje o tym, czy użytkownik zobaczy stronę główną z widokiem kamery (zakodowaną w `index.h`), czy panel logowania (`login.h`), bazując na wcześniejszym sprawdzeniu ciasteczka sesyjnego:

```
// Sprawdzenie Ciasteczek
bool is_authenticated(httpd_req_t *req) {
    size_t buf_len = httpd_req_get_hdr_value_len(req, "Cookie") + 1;
    if (buf_len > 1) {
        char* buf = (char*)malloc(buf_len);
        if (httpd_req_get_hdr_value_str(req, "Cookie", buf, buf_len) == ESP_OK) {
            // Szukanie ciastka o nazwie "auth=1"
            if (strstr(buf, "auth=1") != NULL) {
                free(buf);
                return true;
            }
        }
        free(buf);
    }
    return false;
}

static esp_err_t index_handler(httpd_req_t *req) {
    httpd_resp_set_type(req, "text/html");
    if (is_authenticated(req)) {
        // Zalogowany
        return httpd_resp_send(req, index_html, strlen(index_html));
    } else {
        // Niezalogowany
        return httpd_resp_send(req, login_html, strlen(login_html));
    }
}
```

Mechanizm autoryzacji przetwarza proste żądanie GET przesyłane z przeglądarki. Funkcja `login_handler` odbiera zapytanie, weryfikuje poprawność hasła zdefiniowanego w systemie i jeżeli jest ono prawidłowe, nadaje przeglądarce poświadczenie w postaci pliku Cookie (`auth=1`) ważnego przez 3600 sekund. Następnie wykonuje przekierowanie (Status HTTP 303) na główny ekran:

```
// Handler weryfikacji hasła (odbiera dane z formularza logowania)
static esp_err_t login_handler(httpd_req_t *req) {
    char buf[100];
    if (httpd_req_get_url_query_str(req, buf, sizeof(buf)) == ESP_OK) {
        char pass[50];
        if (httpd_query_key_value(buf, "pass", pass, sizeof(pass)) == ESP_OK) {

            if (strcmp(pass, WEB_PASSWORD) == 0) {
                httpd_resp_set_hdr(req, "Set-Cookie", "auth=1; Path=/; Max-Age=3600");
                httpd_resp_set_status(req, "303 See Other"); // Przekierowanie po sukcesie
                httpd_resp_set_hdr(req, "Location", "/");
                return httpd_resp_send(req, NULL, 0);
            }
        }
    }
    // Bledne haslo odswiezenie strony
    httpd_resp_set_status(req, "303 See Other");
    httpd_resp_set_hdr(req, "Location", "/");
    return httpd_resp_send(req, NULL, 0);
}
```

5. Strumieniowanie obrazu na niezależnym serwerze

Krytycznym punktem wydajnościowym przesyłania obrazu z mikrokontrolera jest uniknięcie blokowania zapytań interfejsu (np. przycisków ruchu `left_handler`, `right_handler`) przez niekończący się strumień obrazu. Zrealizowano to poprzez uruchomienie dwóch oddzielnych instancji serwera HTTP. Serwer interfejsu działa na standardowym porcie 80, natomiast dla strumienia obrazu podnoszony jest dodatkowy serwer (port 81).

Za główne przetwarzanie obrazu odpowiada funkcja `stream_handler`. W pętli nieskończonej pobiera ona klatki z kamery, kompresuje je do standardu JPEG, a następnie fragmentuje i przesyła za pomocą nagłówka typu `multipart/x-mixed-replace` prosto na frontend.

Zastosowano tu obsługę błędów oraz optymalizację polegającą na usypianiu procesu na 50 ms w każdej iteracji, by procesor nadążał z zadaniami pobocznymi:

```
static esp_err_t stream_handler(httpd_req_t * req) {
    // Zabezpieczenie strumienia kamery
    if (!is_authenticated(req)) return httpd_resp_send_err(req,
        HTTPD_401_UNAUTHORIZED, "Brak dostępu");
    if (!camera_ok) {
```

```

    httpd_resp_set_type(req, "text/plain");
    return httpd_resp_send(req, "Camera not available", 20);
}
camera_fb_t * fb = NULL;
esp_err_t res = ESP_OK;
size_t _jpg_buf_len = 0;
uint8_t * _jpg_buf = NULL;
char * part_buf[64];
res = httpd_resp_set_type(req, _STREAM_CONTENT_TYPE);
if (res != ESP_OK) {
    return res;
}
while (true) {
    fb = esp_camera_fb_get();
    if (!fb) {
        Serial.println("Uchwycenie obrazu się nie powiodło");
        continue;
    } else {
        if (fb->width > 400) {
            if (fb->format != PIXFORMAT_JPEG) {
                bool jpeg_converted = frame2jpg(fb, 80, &_jpg_buf, &_jpg_buf_len);
                esp_camera_fb_return(fb);
                fb = NULL;
                if (!jpeg_converted) {
                    Serial.println("JPEG compression failed");
                    res = ESP_FAIL;
                }
            } else {
                _jpg_buf_len = fb->len;
                _jpg_buf = fb->buf;
            }
        }
    }
}
if (res == ESP_OK) {
    size_t hlen = snprintf((char *) part_buf, 64, _STREAM_PART, _jpg_buf_len);
    res = httpd_resp_send_chunk(req, (const char *) part_buf, hlen);
}
if (res == ESP_OK) {
    res = httpd_resp_send_chunk(req, (const char *) _jpg_buf, _jpg_buf_len);
}
if (res == ESP_OK) {
    res = httpd_resp_send_chunk(req, _STREAM_BOUNDARY,
strlen(_STREAM_BOUNDARY));
}
if (fb) {
    esp_camera_fb_return(fb);
    fb = NULL;
}

```

```

    _jpg_buf = NULL;
} else if (_jpg_buf) {
    free(_jpg_buf);
    _jpg_buf = NULL;
}
if (res != ESP_OK) {
    break;
}
//Usypianie na chwile bo juz nie wyrabia
vTaskDelay(pdMS_TO_TICKS(50));
}
return res;
}

```

Dzięki takiemu architektonicznemu podziałowi żądania sterujące mogą być bez problemu odbierane przez główny serwer, nie kolidując z pętlą przetwarzania wizji.

6. Asynchroniczne sterowanie urządzeniem wykonawczym (FreeRTOS)

Aby obsługa silnika serwomechanizmu również nie spowalniała głównego wątku układu, oddelegowano jego obsługę sprzętową do zintegrowanego systemu czasu rzeczywistego (FreeRTOS). Aplikacja sterująca serwem została odizolowana i przypięta na stałe do wykonywania na drugim rdzeniu mikrokontrolera (Rdzeń 1) za pomocą funkcji `xTaskCreatePinnedToCore`.

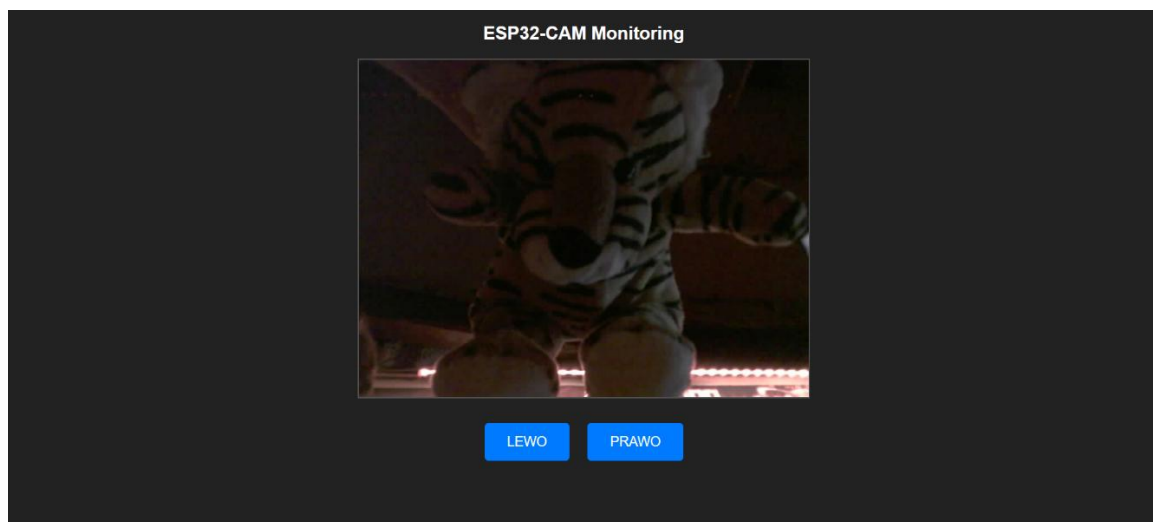
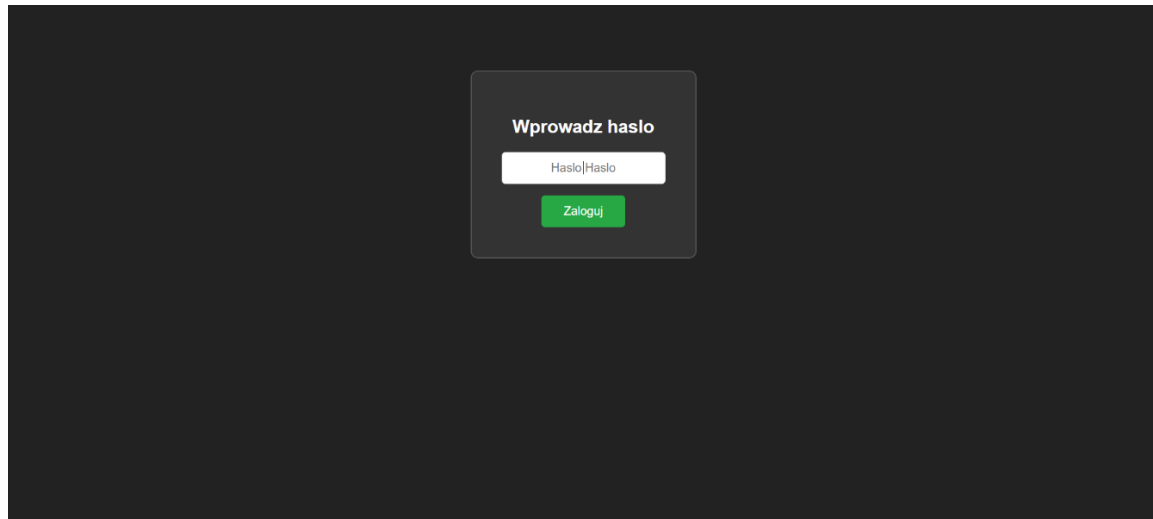
```

void servoTask(void * parameter) {
    for(;;) { // Nieskończona pętla Taska
        if (current_position != target_position) {
            current_position = target_position;
            servo.write(current_position);
            Serial.printf("Serwo rusza na: %d\n", current_position);
        }
        // Usypia wątek na 20ms, zdejmując obciążenie z procesora
        vTaskDelay(pdMS_TO_TICKS(20));
    }
}

```

Akcja użytkownika klikającego przycisk w panelu jedynie aktualizuje zmienną docelową `target_position`. Drugi rdzeń odczytuje tę zmianę asynchronicznie i wymusza fizyczny obrót w tle.

7. Prezentacja działania systemu



8. Podsumowanie

Zrealizowany projekt sprzętowy bazujący na ESP32-CAM udowadnia, że zastosowanie podziału warstwowego oraz asynchroniczności mocno podnosi jakość urządzeń z zakresu IoT. Wrzucenie strumienia wizji na osobny port serwera (port 81) i przetransferowanie obliczeń układu sterowania na oddzielny rdzeń systemu zapewnia płynność aplikacji docelowej dla użytkownika. Dołączenie do urządzenia wstępnej konfiguracji poprzez panel Bluetooth w telefonie czyni urządzenie mobilnym i łatwym we wdrażaniu docelowym u klientów.

9. Link do repozytorium

https://github.com/Redii9/ESP32_Cam